

Configuring and Building Version 13 of Unicon

Jafar Al-Gharaibeh and Clinton Jeffery

Unicon Technical Report: 21

2019-04-16

Abstract

Unicon Version 13 features a fully automatic configuration and build mechanism based on GNU autoconf.

Unicon Project
<http://unicon.org>
University of Idaho
Department of Computer Science
Moscow, ID, 83844, USA

0.0.1 1. Background

The implementation of the Unicon programming language is written mostly in C and RTL [4], a superset of C, for which a translator to C is provided. The Unicon translator itself is written in Unicon, with the C-based legacy Icon translator `icont` used as the code generator and linker. A small amount of assembly-language code is utilized if available for the context switch used by co-expressions. See Appendix B. This code is optional and only affects co-expressions.

There presently are implementations of Unicon only for platforms that host UNIX-compatible shell scripting and configuration tools such as `autoconf`. This includes most versions of UNIX such as Linux, `*BSD` and MacOS, as well as Microsoft Windows 10 via its Ubuntu shell or MSYS and MinGW development environment.

All implementations of Unicon are obtained from the same source code, using conditional compilation and defined constants to select and configure platform-dependent code. Appropriate values for these constants are determined automatically by a `configure` shell script, as is common on UNIX-based software. The `configure` script is run automatically the first time a build is performed, or when any configuration files may have been changed. The `configure` script can also be run manually by typing `./configure` or on windows `sh configure`. A number of features in a Unicon build are optional, selected by command line arguments to the `configure` script. Consequently, installing Unicon on a new platform is largely a matter of selecting configuration options (usually the defaults are adequate), and typing `make`.

The purpose of this document is to describe aspects of the process of configuring Version 13 of the Unicon source code, in the event that optional arguments are needed or if `make` does not work out of the box. For example, `make` might not work on a platform on which Unicon has not previously been installed.

Building Unicon with a new C compiler on an operating system where Unicon has previously been installed may be a fairly simple task and ideally will be handled by the `configure` script. Porting Unicon to a new operating system or to an environment in which the `configure` script does not run is more complex; read this report carefully before undertaking such a project.

0.0.2 2. Requirements

C Data Sizes Unicon places the following requirements on C data sizes:

- `char` must be 8 bits.
- `int` must be 32 or 64 bits.
- `long` and pointers must be 32 or 64 bits.
- All pointers must be the same length.
- `long` and pointers should be the same length.

If your C data sizes do not meet these requirements, do not expect Unicon to build. If `long` and pointers are not the same length, such as the case on Windows, new data type definitions might be needed to ensure the runtime uses the correct data type sizes.

The C Compiler The main requirement for implementing Unicon is a production-quality C compiler that supports ANSI C. The term "production quality" implies robustness, correctness, the ability to address large amounts of memory, the ability to handle large files and complicated expressions, and a comprehensive run-time library.

Memory The Unicon programming language requires a substantial amount of memory to run. The practical minimum depends somewhat on the platform, but the minimum would be measured in megabytes, not kilobytes or gigabytes.

File Space The source code for Unicon is several megabytes. Test programs and other auxiliary files take additional room, as does compilation and testing. While the implementation can be divided into components that can be built separately, this approach may be painful.

0.0.3 3. File Structure

The files for Unicon are organized in a hierarchy. The top level, assuming the hierarchy is rooted in `unicon` is:

	-bin-----	executable binaries and support files
	-config---	configurations
-unicon--	-src-----	source code
	-tests----	tests
	-uni-----	unicon

There are several subdirectories in `config`. Historically this directory contained support for different operating systems and compilers. At this point it is largely vestigial.

	-editor---	
	-port-----	
--config--	-scripts--	
	-unix-----	
	-win32----	

| **Some configuration directories contain subdirectories for different platforms.** These subdirectories contain various files, depending on the platform.

| **The directory “src“ contains the source code for various components of Unicon.**

	-asm-----	assembler (co-expression) source
	-common----	common source
	-gdbm-----	GDBM local indexed file library source
	-h-----	header files
	-iconc-----	Optimizing compiler source
-src-----	-icont-----	VM bytecode translator source
	-lib-----	libraries
	-preproc---	C preprocessor source
	-rtt-----	run-time translator source
	-runtime---	run-time support
	-xpm-----	XPM image-file format support

The directory `tests` contains the test material for various components of Unicon.

	-bench----	benchmarks
	-calling---	calling C functions from Unicon
	-coexpr----	co-expressions
	-container-	container
	-general---	general tests
	-graphics--	graphics facilities
	-ipl-----	Icon program library
	-lib-----	library
-tests----	-mt-----	multi-thread?
	-pattern---	pattern matching
	-posix----	POSIX facilities
	-preproc---	C preprocessor tests
	-samples---	short sample programs
	-special---	special features
	-thread----	threads
	-udb-----	debugger
	-unicon----	unicon language features

The directory `uni` contains source code for the Unicon translator and class libraries. Code in these subdirectories is generally written in Unicon.

	-3d-----	3D graphics class libraries
	-cint-----	C interface support
	-gprogs----	graphics programs
	-gui-----	graphical user interface class library
	-ide-----	ui integrated development environment
	-ivib-----	version 1 improved visual interface builder (deprecated)
	-iyacc-----	Icon yacc, a parser generator
	-lib-----	general purpose Unicon libraries
	-monvis----	monitoring and visualization tools
	-native----	native something
-uni-----	-parser----	a unicon parser library
	-progs----	programs
	-shell----	a simple shell
	-udb-----	Unicon debugger
	-ulex-----	Unicon lexical analyzer generator
	-unicon----	Unicon translator
	-unidep----	Unicon dependency generator
	-unidoc----	Unicon documentation generator

-util-----	utilities
-xml-----	XML library

The Unicon optimizing compiler (`unicon -C`) requires the presence of a C compiler and is therefore not included in all distributions of Unicon.

0.0.4 4. Parameters and Definitions

| **There are many defined constants and macros in the source code for**

Unicon that vary from platform to platform. Over the range of possible platforms, there are many possibilities. A list is given in Appendix A. *Do not be intimidated by the large number of options listed there;* most are provided only for unusual situations and only a few are needed for any one platform.

| **Historically, the defined constants and macros needed for a specific**

platform were placed manually in `src/h/define.h`. On reasonable modern platforms, this has all been done away with and corresponding defines are automatically determined by the `configure` script and placed in `src/h/auto.h` using the placeholder and input file `src/h/auto.h.in`. In addition to the C language definitions placed in `src/h/auto.h`, the `configure` script also generates symbols and environment variable definitions which are used during the build process or in some cases control the behavior of the build itself. These additional files are all placed in the top level directory and include:

`Makefile` updated from `Makefile.in` `Makedefs` generated from `Makedefs.in`. It holds parameters and symbols used with the C language compiler.
`Makedefs.uni` generated from `Makedefs.uni.in`. It holds parameters symbols used with the Unicon language translator and compiler.

In the unfortunate event that you need to build Unicon on a system where the `configure` script does not run, `src/h/define.h` can still be used and you can still manually copy in or tweak makefiles or the `Makedefs` files and such, however this should be considered a last resort. An example `src/h/define.h` for a "vanilla" 32-bit platform is:

```
#define HostStr "new host"
#define NoCoexpr
#define PORT 1
```

`HostStr` provides the value used in the Unicon keyword `&host` and should be changed as appropriate. `NoCoexpr` causes Unicon to be configured without co-expressions. This definition can be removed when co-expressions are implemented. See Appendix B. `PORT` indicates an implementation for an unspecified operating system. It should be changed to a name for the operating system for the new platform (see Section 7). Other definitions probably need to be added, of course.

0.0.5 5. Configuring Unicon for a UNIX Platform

Unicon has been implemented for many UNIX platforms; support for the POSIX standard (see Reference 4) is expected. `make` might just do the right thing, running the `configure` script and then building the Unicon translators and runtime system. The full configure+build sequence is:

```
./configure
make
```

The biggest issue with building Unicon on a new machine is usually the question of what optional language features are required, and what packages of C libraries and header files must be built or installed in order for Unicon to provide those features.

5.1 Configure Arguments The Unicon configure script will enable many features that it finds libraries and header files for, by default. A summary of the enabled and disabled features is given at the end of the configuration script. Some experimental features are not part of the Unicon language canon and not turned on by default, but can be turned on from the configure script. Disabling unwanted features, or enabling non-canon features, is accomplished by command line arguments to `configure`. The command

```
./configure --help
```

lists these available arguments. An assortment of them are given below for illustrative purposes:

```
--disable-graphics      No graphics subsystem
--disable-graphics3d    No 3D graphics support
```

<code>--disable-concurrency</code>	No concurrent thread support
<code>--disable-pattern</code>	No pattern type support
<code>--disable-database</code>	No database support
<code>--disable-ssl</code>	No SSL support
<code>--disable-audio</code>	No audio support
<code>--disable-voip</code>	No VOIP support
<code>--disable-plugins</code>	No loadfunc or plugins support
<code>--disable-iconc</code>	Build Unicon Compiler (Iconc/Uniconc)
<code>--enable-iconcurrency</code>	enable thread support in Unicon Compiler
<code>--enable-ovld</code>	enable operator overloading
<code>--enable-udbtools</code>	enable Unicon debugger tools
<code>--enable-progs</code>	enable Unicon programs
<code>--enable-verbosebuild</code>	Show full CC build lines with all compiler arguments
<code>--enable-thin</code>	Do a minimalist build disabling non critical features

Similarly, many optional libraries may be located in interesting non-default locations, which are specified via arguments to `configure` such as:

<code>--with-zlib[=DIR]</code>	Use zlib package (DIR: custom library path)
<code>--with-xlib[=DIR]</code>	Use xlib package (DIR: custom library path)
<code>--with-freetype[=DIR]</code>	Use freetype package (DIR: custom library path)
<code>--with-Xft[=DIR]</code>	Use Xft package (DIR: custom library path)
<code>--with-jpeg[=DIR]</code>	Use jpeg package (DIR: custom library path)
<code>--with-png[=DIR]</code>	Use png package (DIR: custom library path)
<code>--with-opengl[=DIR]</code>	Use opengl package (DIR: custom library path)
<code>--with-ftgl[=DIR]</code>	Use ftgl package (DIR: custom library path)
<code>--with-ogg[=DIR]</code>	Use ogg package (DIR: custom library path)
<code>--with-SDL[=DIR]</code>	Use SDL package (DIR: custom library path)
<code>--with-smpeg[=DIR]</code>	Use smpeg package (DIR: custom library path)
<code>--with-openal[=DIR]</code>	Use openal package (DIR: custom library path)
<code>--with-jvoip[=DIR]</code>	Use jvoip package (DIR: custom library path)
<code>--with-odbc[=DIR]</code>	Use odbc package (DIR: custom library path)
<code>--with-pthread[=DIR]</code>	Use pthread package (DIR: custom library path)
<code>--with-ssl[=DIR]</code>	Use ssl package (DIR: custom library path)

5.2 Example: Enabling Graphics Facilities On UNIX systems that run X Windows, you may wish to configure Unicon with X support. Uni-

con's graphics facilities call Xlib, the standard C interface to X. At present, configuration of X Window facilities is provided only for UNIX platforms.

In order to build Unicon with these X Window functions, you will need to know what library or libraries are required to link in the X facilities into C programs; this library information is needed when `iconx` is built and when `iconc` links a compiled Unicon executable. Normally, the answer will be `-lX11`, but on some platforms additional libraries or alternate paths are required. Consult appropriate manuals to find out what libraries are needed.

The `configure` will find the X11 library if your platform has it at one of the most common locations. If not, there are two possibilities. If it is installed at an unusual location, you may end up using a command line option to configure to specify its location:

```
./configure --with-xlib=/my/unusual/location
```

In a more extreme case, you may have to install the development libraries and header include files appropriate for C language X11 development on your system.

Historically, the files `xiconx.mak` and `xiconc.def`, if they are present, were used during Unicon configuration to supply non-default library information to the interpreter and the compiler. Although these files are not normally used any more, the description below is retained if it is needed on some non-UNIX build at some point.

If a platform requires an additional pseudo-terminal library and a BSD-compatibility package in order to link X applications, you would edit the `Makedefs` top level files and change the `XLIB` line to include the new `-lbsd` library as follows:

```
XLIB= -L../bin -lX11 -lpt -lbsd
```

Historically, and possibly still, there was a corresponding `xiconc.def` file that took a line such as

```
#define ICONC XLIB "-lX11 -lpt -lbsd"
```

| **The former (`XLIB=...`) line gets prepended to the flags passed to the C compiler/linker when building `iconx`, while the latter file gets included and compiled into `iconc` when X is configured. Then proceed to the `make` build step.**

| **In order to build Unicon with X support, some platforms also will have** to specify the location of the X header files. Normally they are in `/usr/include/X11`; if they are in some other place on your platform, you will need to locate them and identify the appropriate option to add to the C compiler command line, usually `-I path`, where *path* is the directory above the X11 include directory.

For the Unicon compiler, this option is added via the `C0pts` macro in `define.h` for your configuration. The `C0pts` macro must define a quoted C string. For the interpreter, the option is added to the `CFLAGS` argument of the `common.hdr`, `icont.hdr`, `runtime.hdr`, and `xpm.hdr` Makefile headers for your configuration.

0.0.6 6. Configuring Unicon for an MS Windows Platform

In the case of Windows, the primary considerations in configuring Unicon have to do with the C compiler that is used. Historically Icon ran on many 16- to 64-bit DOS and Windows compilers. At present, Unicon is known to build on Windows using either

- the 64-bit Mingw version of GCC, when the version 1 of MSYS UNIX-like tools and shell are installed.
- the Windows Ubuntu shell allows a Unix-like Unicon to be built that runs well other than known issues in the threads facilities.

For the 64-bit Mingw version of GCC, the configuration and build step is performed by invoking

```
make WUnicon64
make
```

where the first line is an alias for

```
sh configure --build=x86_64-w64-ming32 --disable-iconc
```

This begs the question of how to build Unicon's `iconc` on Windows. Undoubtedly, many Bothan spies will die to bring us this information. A Windows `iconc` port was performed awhile back, but building it does not occur by default.

The rest of this section should speculate on what it would take to build Unicon using the last-known-but-currently-unsupported Windows Compiler, Microsoft's C/C++ compiler.

Unicon's autoconf-based configuration script, `configure`, has not been tested as to whether it would find or use a Microsoft C compiler if it was on the path. It is thus expected that a traditional Icon-style manual configuration process would be required. Feel free to update the files in the `config/win32/msvc/` directory for us and have a go at it.

An MS Windows configuration includes `Makefile\ s`, batch scripts, and response files for linking. These files should be modified for the new compiler as appropriate. According to `config/win32/msvc/status`, the last time MSVC was used in a built was 2014 under Visual C++ 18.0 from Visual Studio 12.0.

0.0.7 7. Configuring Unicon for a New Operating System

The conditional compilation for specific operating systems is concerned primarily with matters such as differences in file naming, the handling of input and output, and environmental factors. Conditional compilation uses logical expressions composed from these symbols. An example is:

```
...
#if NT
...
/* code for MS Windows */
...
#endif
#if UNIX || VMS
...
/* code for UNIX and VMS */
...
#endif
...
```

Each symbol is defined to be either 1 (for the target operating system) or 0 (for all other operating systems). This is accomplished by defining the symbol for the target operating system to be 1 in `define.h`. In `config.h`, which includes `define.h`, all other operating-system symbols are defined to be 0.

| **Logical conditionals with “#if” are used instead of defined or undefined** names with `#ifdef` to avoid nested conditionals, which become very complicated and difficult to understand when there are several alternative operating systems. Note that it is important not to use `#ifdef` in place of `#if`, since all the names are defined.

| **The file “define.h” for a different operating system should** initially contain

```
::  
#define PORT 1
```

as indicated in Section 4. You can use `PORT` during the configuration for a different operating system. Later you should come back and change `PORT` to some more appropriate name.

| **Note: The “PORT” sections contain deliberate syntax errors (so** marked) to prevent sections from being overlooked during configuration. These syntax errors must, of course, be removed before compilation.

| **To make it easy to locate places where there is code that may be** dependent on the operating system, such code usually is bracketed by unique comments of the following form:

```
::  
/ The following code is operating-system dependent. / ... / End of  
operating-system specific code. /
```

Between these beginning and ending comments, the code for different operating systems is provided using conditional expressions such as those indicated above.

Look through some of the files for such segments to get an idea of what is involved. Each segment contains comments that describe the purpose of the code. In some cases, the most likely code or a suggestion is given in the conditional code under `PORT`. In some cases, no code will be needed. In others, code for an existing operating system may suffice for the new one.

In any event, code for the new operating system name must be added to each such segment, either by adding it to a logical disjunction to take advantage of existing code for other operating systems, as in

```
#if MSDOS || UNIX || PORT
...
#endif

#if VMS
...
#endif
```

and removing the present code for PORT or by filling in the segment with the appropriate code, as in

```
#if PORT
...
/* code for the new operating system */
...
#endif
```

- | **If no code is needed for the target operating system in a particular** situation, a comment should be provided so that it is clear that the situation has been considered.
- | **You may find need for code that is operating-system dependent at a** place where no such dependency presently exists. If the situation is idiosyncratic to your operating system, which is most likely, simply use a conditional for PORT as shown above. If the situation appears to need different code for several operating systems, add a new segment similar to the other ones, being sure to provide something appropriate for all operating systems.
- | **Do not use “#else“ constructions in these segments; this increases** the probability of logical errors and obscures the mutually exclusive nature of operating system differences.

0.0.8 8. Trouble Reports and Feedback

If you run into problems, contact us at the Unicon Project:

Unicon Project, c/o Clinton Jeffery Department of Computer Science The University of Idaho 875 Perimeter Drive Moscow, ID 83844-1010 U.S.A. (208) 885-4789 (voice)

Please also let us know of any suggestions for improvements to the configuration process.

Once you have completed your installation, please send us copies of any files that you modified so that we can make corresponding changes in the central version of the source code. Once this is done, you can get a new copy of the source code whenever changes or extensions are made to the implementation. Be sure to include documentation on any features that are not implemented in your installation or any changes that would affect users.

References

- | 1. **Clinton Jeffery and Donald Ward, editors, *The Implementation of the Icon and Unicon: a Compendium***, unicon.org/book/ib.pdf.
- | 2. **B. W. Kernighan and D. M. Ritchie, *The C Programming Language***, Prentice-Hall, Inc., Englewood Cliffs, NJ, first edition, 1978.
- | 3. ***American National Standard for Information Systems – Programming Language - C, ANSI X3.159-1989***, American National Standards Institute, New York, 1990.
- | 4. ***IEEE Standard 1003.1-1988, Portable Operating System Interface for Computer Environments*** ("POSIX .1"), Institute of Electrical and Electronics Engineers, New York, 1988.

0.0.9 Appendix A – Configuration Parameters and Definitions

C Compiler Considerations On some platforms it may be necessary to provide a different *typedef* for pointer than is provided by default. For example, one old-timey version of Microsoft C used a `define.h` with the following in it:

```
typedef huge void *pointer;
```

If an alternative *typedef* is used for pointer, add

```
#define PointerDef
```

to `define.h` to avoid the default one.

Sometimes computing the difference of two pointers causes problems. Pointer differences are computed using the macro `DiffPtrs(p1, p2)`, which has the default definition:

```
#define DiffPtrs(p1, p2) (word)((p1)-(p2))
```

where `word` is a *typedef* that is provided automatically and usually is *long int*.

This definition can be overridden in `define.h`. For example, at one time Microsoft C used

```
#define DiffPtrs(p1, p2) ((word)(p1)-(word)(p2))
```

If you provide an alternate definition for pointer differencing, be careful to enclose all arguments in parentheses.

Character Set The default character set for Unicon is ASCII. If you are configuring Unicon for a platform that uses the EBCDIC character set, add

```
#define EBCDIC 1
```

to `define.h`.

Data Sizing and Alignment There are two constants that relate to the size of C data:

```
WordBits    (default: 32)
IntBits     (default: WordBits)
```

`IntBits` is the number of bits in a C *int*. It may be 16, 32, or 64. `WordBits` is the number of bits in a C *long* (Unicon's "word"). It may be 32 or 64.

If your C library expects *doubles* to be aligned at double-word boundaries, add

```
#define Double
```

to `define.h`.

The word alignment of stacks used by co-expressions is controlled by

```
StackAlign    (default: 2)
```

If your platform needs a different alignment, provide an appropriate definition in `define.h`.

Most computers have downward-growing C stacks, for which stack addresses decrease as values are pushed. If you have an upward-growing stack, for which stack addresses increase as values are pushed, add

```
#define UpStack
```

to `define.h`.

Floating-Point Arithmetic There are three optional definitions related to floating-point arithmetic:

```
Big           (default: 9007199254740092.)  
LogHuge       (default: 309)  
Precision     (default: 10)
```

The values of `Big`, `LogHuge`, and `Precision` give, respectively, the largest floating-point number that does not lose precision, the maximum base-10 exponent $+ 1$ of a floating-point number, and the number of digits provided in the string representation of a floating-point number. If the default values given above do not suit the floating-point arithmetic on your platform, add appropriate definitions to `define.h`.

Large Integers Large-integer arithmetic is normally enabled. Because this feature increases the size of the run-time system by 15-20%, it may be necessary to disable it on computers with limited memory. To do this, add

```
#define NoLargeInts
```

to `define.h`.

Storage Region Sizes The default sizes of Unicon’s run-time storage regions for allocated data normally are calculated from the amount of physical memory on the machine. However, different values can be set:

MaxAbrSize	(default: 2% of available memory)
MaxStrSize	(default: 2% of available memory)

Since users can override the set values with environment variables, it is unwise to change them from their defaults except in unusual cases.

The sizes for Unicon’s main interpreter stack and co-expression stacks also can be set:

MStackSize	(default: 0.5% of available memory)
StackSize	(default: 0.02% of available memory)

As for the block and string storage regions, it is unwise to change the default values except in unusual cases.

Finally, a list used for pointers to strings during garbage collection, can be sized:

QualLstSize	(default: 5000)
-------------	-----------------

This one normally is best left unchanged. The qualifier list gets reallocated (and doubled in size) whenever it is found to be not large enough.

Allocation Sizing *Note: the following discussion is true but obsolete. It is retained for historical reasons and may be of some small use to embedded systems and retro computing enthusiasts.*

`malloc()` is used to allocate space for Unicon’s storage regions. This limits region sizes to the value of the largest unsigned int. Historically, some platforms provided alternative allocation routines for allocating larger regions. To change the allocation procedure for regions, add a definition for `AllocReg` to `define.h`. For example, an ancient huge-memory-model implementation of Icon for Microsoft C used the following:

<code>#define AllocReg(n) halloc((long)n, sizeof(char))</code>
--

Note: Unicon still uses `malloc()` for allocating other blocks. If this is a problem, it may be possible to change this by defining `malloc` in `define.h`, as in

```
#define malloc lmalloc
```

where `lmalloc()` is a local routine for allocating large blocks of memory. If this is done, and the size of the allocation is not *unsigned int*, add an appropriate definition for the type by defining `AllocType` in `define.h`, such as

```
#define AllocType unsigned long int
```

It is also necessary to add a definition for the limit on the size of a Unicon region:

```
#define MaxBlock n
```

where `n` is the maximum size allowed (the default for `MaxBlock` is `MaxUnsigned`, the largest *unsigned int*). It generally is not advisable to set `MaxBlock` to the largest size an alternative allocation routine can return. For the huge-memory-model implementation mentioned above, `MaxBlock` is 256000.

File Name Suffixes The suffixes used to identify source programs, ucode object files, and icode binary program files may be specified in `define.h`:

```
#define SourceSuffix (default: ".icn")
#define U1Suffix      (default: ".u1")
#define U2Suffix      (default: ".u2")
#define USuffix       (default: ".u")
#define IcodeSuffix   (default: "")
#define IcodeASuffix  (default: "")
```

| “**USuffix**“ is used for ***ucode* files that are Unicon’s combined** assembler/object file format. Ucode files are formed from two temporary files that use the `U1Suffix` and `U2Suffix`, respectively. `IcodeASuffix` is an alternative suffix that `iconx` uses when searching for icode files specified without a suffix. For example, on a Windows system, a non-exe-bundled binary bytecode file might adopt the `IcodeSuffix` `".cmd"` or the case-insensitive alternative `IcodeASuffix` is `".CMD"`.

| **If values other than the defaults are specified, care must be taken**
not to introduce conflicts or collisions among names of different types
of files.

Paths If `icont` is given a source program in a directory different from the local one ("current working directory"), there is a question as to where ucode and icode files should be created: in the local directory or in the directory that contains the source program. On most platforms, the appropriate place is in the local directory (the user may not have write permission in the directory that contains the source program). However, on some platforms, the directory that contains the source file is appropriate. By default, the directory for creating new files is the local directory. The other choice can be selected by adding

```
#define TargetDir SourceDir
```

Command-Line Options The command-line options that are supported by `icont` and `iconc` are defined by `IconOptions`. The default value (see `config.h`) will do for most platforms, but an alternative can be included in `define.h`.

Similarly, the error message produced for erroneous command lines is defined by `TUsage` for `icont` and `CUsage` for `iconc`. The default values, which should correspond to the value of `IconOptions`, are in `config.h`, but may be overridden by definitions in `define.h`.

If your C library includes `getopt()`, you can add

```
#define SysOpt
```

to use the library function instead of Unicon's private version.

Host Identification If your system does not include a `uname()` library function, the value of the Unicon keyword `&host` must be specified by adding

```
#define HostStr "identification"
```

to `define.h`.

Directory Reading If your platform supports the `opendir()` and `readdir()` functions for reading directories, add

```
#define ReadDirectory
```

to `define.h`.

Keyboard Functions If your platform supports the keyboard functions `getch()`, `getche()`, and `kbhit()`, add

```
#define KeyboardFncs
```

to `define.h`.

You can also define `KeyboardFncs` if you supply your own keyboard functions; see `src/runtime/rlocal.r` for examples.

Dynamic Loading If your platform supports the `dlopen()` and `dlsym()` functions for dynamic loading, add

```
#define LoadFunc
```

to `define.h`.

Co-Expressions The implementation of co-expressions requires an assembly-language context switch. If your platform does not have a co-expression context switch, you can implement one as described in Appendix B. Alternatively, you can disable co-expressions by adding

```
#define NoCoexpr
```

to `define.h`.

X Window Facilities The files needed to build Unicon with X Window facilities are not in the same places on all platforms. If Unicon fails to build because an include file needed by X cannot be found, it may be necessary to edit `src/h/sys.h` to reflect the local location.

Some early versions of X Window Systems, notably X11R3, do not support the attribute `iconic`. If this is the case for your platform, add

```
#define NoIconify
```

to `define.h`. This disables the attribute `iconic`, causing references to it to fail.

Compiler Options The C compiler called by the Icon compiler, \ `iconc`, to process its output defaults to `cc`. If you want to use a different C compiler, add

```
#define CComp "name"
```

to `define.h`, where “*name*” is the name of the C compiler you want the Icon compiler to use. Note the quotation marks surrounding the name. For example, to use Gnu C, add

```
#define CComp "gcc"
```

By default, the C compiler is called with no options. If you want specific options, add

```
#define COpts "options"
```

to `define.h`. Again, note the quotation marks. For example, to request C optimizations, you might add

```
#define COpts "-O"
```

If your system does not have `ranlib`, add

```
#define NoRanlib
```

to `define.h`.

Dynamic Hashing Constants Four parameters configure the implementation of tables and sets:

HSlots Initial number of hash buckets; it must be a power of 2 **HSegs** Maximum number of hash bucket segments **MaxHLoad** Maximum allowable loading factor **MinHLoad** Minimum loading factor for new structures

The default values (listed below) are appropriate for most platforms. If you want to change the values, read the discussion that follows.

Every set or table starts with **HSlots** hash buckets, using one bucket segment. When the average hash bucket exceeds **MaxHLoad** entries, the number of buckets is doubled and one more segment is consumed. This repeats until **HSegs** segments are in use; after that, structure still grows but no more hash buckets are added.

MinHLoad is used only when copying a set or table or when creating a new set through the intersection, union, or difference of two other sets. In these cases a new set may be more lightly loaded than otherwise, but it is never less than **MinHLoad** if it exceeds a single bucket segment.

For all machines, the default load factors are 5 for **MaxHLoad** and 1 for **MinHLoad**. Because splitting or combining buckets halves or doubles the load factor, **MinHLoad** should be no more than half **MaxHLoad**. The average number of elements in a hash bucket over the life of a structure is about $(2/3)*\text{MaxHLoad}$, assuming the structure is not so huge as to be limited by **HSegs**. Increasing **MaxHLoad** delays the creation of new hash buckets, reducing memory demands at the expense of increased search times. It has no effect on the memory requirements of minimally-sized structures.

HSlots and **HSegs** interact to determine the minimum size of a structure and its maximum efficient capacity. The size of an empty set or table is directly related to **HSegs+HSlots**; smaller values of these parameters reduce the memory needs of programs using many small structures. Doubling **HSlots** delays the onset of the first structure reorganization until twice as many elements have been inserted. It also doubles the capacity of a structure, as does increasing **HSegs** by 1.

The maximum number of hash buckets is $\text{HSlots}*(2^{(\text{HSegs}-1)})$. A structure can be considered "full" when it contains **MaxHLoad** times that many entries; beyond that, lookup times gradually increase as more elements are added. Until a structure becomes full, the values of **HSlots** and **HSegs** do not affect lookup times.

For machines with 16-bit *ints*, the defaults are 4 for **HSlots** and 6 for **HSegs**. Sets and tables grow from 4 hash buckets to a maximum of 128,

and become full at 640 elements. For other machines, the defaults are 8 for HSlots and 10 for HSegs. Sets and tables grow from 8 hash buckets to a maximum of 4096, and become full at 20480 elements.

Implementation Debugging Code Separate from user debugger tools such as udb, Unicon contains some code to assist in debugging the implementation. It is enabled by the definitions

```
#define DeBugTrans /* debugging code for the translator in iconc */
#define DeBugLinker /* debugging code for the linker in iconc */
#define DeBugIconx /* debugging code for the run-time */
```

All three of these are automatically defined if DeBug is defined.

The debugging code for the translator consists of functions for dumping symbol tables (see `iconc/tsym.c`). These functions are rarely needed and there are no calls to them in the source code as it is distributed.

The debugging code for the linker consists of a function for dumping the code region (see `iconc/lcode.c`) and code for generating a debugging file that is a printable image of the icode file produced by the linker. This debugging file, which is produced if the option `-L` is given on the command line when `iconc` is run, may be useful if icode files are incorrect.

The debugging code for the executor consists of a few validity checks at places where problems have been encountered in the past. It also provides functions for dumping Unicon values. See `runtime/rmisc.r` and `runtime/rmemmgt.r`.

When installing Unicon on a new operating system, it is advisable to enable the debugging code until Unicon is known to be running properly. The code produced is innocuous and adds only a few percent to the size of the executable files. It should be removed by deleting the definition listed above from `define.h` as the final step in the implementation for a new operating system.

0.0.10 Appendix B – Implementing a Co-Expression Context Switch

If your platform does not have a co-expression context switch, you can implement one as described in this appendix. Note: If your platform does not allow the C stack to be at an arbitrary place in memory, there is probably little hope of implementing co-expressions.

The routine `coswitch()` is needed for context switching. This routine requires assembly language, since it must manipulate hardware registers. It either can be written as a C routine with `asm` directives or directly as an assembly language routine.

Calls to the context switch have the form `coswitch(old_cs,new_cs,first)`, where `old_cs` is a pointer to an array of words (*C longs*) that contain C state information for the current co-expression, `new_cs` is a pointer to an array of words that hold C state information for a co-expression to be activated, and `first` is 1 or 0, depending on whether or not the new co-expression has or has not been activated before. The zeroth element of a C state array always contains the hardware stack pointer (`sp`) for that co-expression. The other elements can be used to save any C frame pointers and any other registers your C compiler expects to be preserved across calls.

The default size of the array for saving the C state is 15. This number may be changed by adding

```
#define CStateSize n
```

| to `define.h`, where `n` is the number of elements needed. | The first thing `coswitch` does is to save the current pointers and registers in the `old_cs` array. Then it tests `first`. If `first` is zero, `coswitch` sets `sp` from `new_cs[0]`, clears the C frame pointers, and calls `new_context`. If `first` is not zero, it loads the (previously saved) `sp`, C frame pointers, and registers from `new_cs` and returns. | Written in C, `coswitch` has the form:

```
/*
 * coswitch
 */
coswitch(old_cs, new_cs, first)
long *old_cs, *new_cs;
int first;
{
    ...
    /* save sp, frame pointers, and other registers in old_cs */
    ...
    if (first == 0) { /* this is first activation */
        ...
        /* load sp from new_cs[0] and clear frame pointers */
    }
}
```



```

        ...
        new_context(0, 0);
        syserr("new_context() returned in coswitch");
    }
else {
    ...
    /* load sp, frame pointers, and other registers from new cs */
    ...
}
}

```

After you implement `coswitch`, remove the `#define NoCoexpr` from `define.h`. Verify that `StackAlign` and `UpStack`, if needed, are properly defined.

| **To test your context switch, run the programs in `tests/general/coexpr.lst`.** Ideally, there should be no differences in the comparison of outputs.

| **If you have trouble with your context switch, the first thing to do is** double-check the registers that your C compiler expects to be preserved across calls – different C compilers on the same computer may have different requirements.

Another possible source of problems is built-in stack checking. Co-expressions rely on being able to specify an arbitrary region of memory for the C stack. If your C compiler generates code for stack probes that expects the C stack to be at a specific location, you may need to disable this code or replace it with something more appropriate.

0.0.11 Appendix C – Obtaining Packages to Enable Optional Unicorn Features

Linux Binary packages for specific development libraries are usually available. Packages are installed using tools whose names have changed over time, and often require super-user privileges. For example, on Ubuntu and other Debian-based Linux distributions, you may need to say

```
sudo apt-get install lib
```

while on Fedora and other Redhat-based Linux distributions you might say

```
sudo dnf install lib
```

category
Ubuntu/Mint/Debian
Fedora/Redhat/Centos
graphics
libx11-dev
libX11-devel
libjpeg-dev
libjpeg-devel
libpng-dev
libpng12-devel
3d
libglu1-mesa-dev
mesa-libGL-devel, mesa-libGLU-devel
libxft-dev
libXft-devel
libfreetype6-dev
libftgl-dev
audio
libopenal-dev
openal-devel
libalut-dev
freealut-devel
libogg-dev
libogg-devel
libvorbis-dev
libvorbis-devel
database
unixodbc-dev
unixODBC-devel
web
libssl-dev
openssl-devel

Windows Windows Unicon is built with Mingw64, for which many of the optional libraries described above are not easily available in binary form. Some of them can be built from source code successfully. The known-successful libraries, where to obtain them, and how to compile them should be listed here eventually, feel free to pester the authors if you need specific functionality on Windows.

MacOS MacOS has its own third-party software sites. For example, the X Window System was a part of MacOS for awhile, but is now relegated to a large external optional download. How to obtain, install or compile optional third party libraries for MacOS should be listed here eventually, feel free to pester the authors if you need specific functionality on MacOS.